

ECS401: Unit 10 Lab Progress Test

FEEDBACK: Answers and Explanations

Work hard and you will eventually be able to program. It is really important that you understand why the answers here are what they are. If anything is confusing then get someone to explain.

Consider the following bubblesort method where the method swap swaps values in the array at the given positions when answering these questions:

```
public static void bubblesort(int[] array)
{
    int n = array.length;
    for(int pass=0; pass <= n-2; pass++)
    {
        for(int pos= 0; pos <= n-2-pass; pos++)
        {
            if (array [pos] > array [pos + 1])
                swap (array, pos, pos + 1);
        }
    }
}
```

1. If given an array of length 450, how many passes will bubblesort need to guarantee to sort the data in to order?

CORRECT ANSWER:

449

EXPLANATION:

To guarantee to sort data bubble sort needs one less pass than the amount of data being sorted, so if there are 450 elements it will take 449 passes. This is because each pass guarantees at least one more element is put into the right place. However, the very last pass puts the last two elements in to the right place because if all but one are right, there is no where else but the right place for the last one to go.

2. How many comparisons will be done on the very **first** pass if this bubblesort algorithm is given an array of length 450?

CORRECT ANSWER:

449

EXPLANATION:

The number of comparisons on the first pass of bubblesort is also one less than the number of elements. The algorithm goes to each position and compares the value

there with the one next to it. However, the last position has nothing to compare with so that one does not lead to a comparison.

In terms of the code, n is 450 (the length of the array, the value of `pass` is 0 (as it counts from 0)), the inner loop is running from 0 to $n-2-\text{pass}$ so to $450-2-0$ so 448. As the test is $\leq$ the inner loop runs 449 times (as counting from 0).

3. How many comparisons will be done on the very **last** pass if this bubblesort algorithm is given an array of length 450?

CORRECT ANSWER:

1

EXPLANATION:

In the naive version of the algorithm, every pass does the same number of comparisons as the first pass. The efficient version of the algorithm as above however does one less comparison each time. By the last pass only the first and second elements of the array are not already guaranteed to be in the right place, so a single comparison to compare them (and swap them or not) is needed to finish the sort.

In terms of the code, n is 450 (the length of the array, the value of `pass` is 448 [as it counts from 0]), the inner loop is running from 0 to $n-2-\text{pass}$ so to $450-2-448$ so 0. As the test is $\leq$ the inner loop runs once.

4. Assuming the above algorithm, what is the sequence of states the array passes through during the first pass if given the array $a = \{4, 3, 1, 6, 9, 5\}$?

CORRECT ANSWER:

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
4	3	1	6	9	5
3	4	1	6	9	5
3	1	4	6	9	5
3	1	4	6	9	5
3	1	4	6	9	5
3	1	4	6	5	9

EXPLANATION:

Notice how the biggest value found so far is dragged along towards the end.

This covers only the first pass and this one pass is not enough to guarantee that the array is sorted.

5. What is the state of the array AFTER each pass of bubblesort is completed if given the initial array `a = {6, 9, 3, 5, 4, 1}`?

CORRECT ANSWER:

PASS	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
START	6	9	3	5	4	1
0	6	3	5	4	1	9
1	3	5	4	1	6	9
2	3	4	1	5	6	9
3	3	1	4	5	6	9
4	1	3	4	5	6	9

6. What would the line

```
if (array [pos] > array [pos + 1])
```

be changed to if the array was to be sorted in to descending order rather than ascending order?

CORRECT ANSWER:

```
if (array [pos] < array [pos + 1])
```

EXPLANATION:

We just need to change the greater than operator for a less than operator and it will now only move them if less than the one next to them so moving smaller ones to the end not larger ones.

Another way would be to leave the operator alone and change instead what is checked i.e.

```
if (array [pos+1] > array [pos])
```

7. Give a version of the algorithm to sort String arrays in to alphabetical order. You may assume you have an appropriate swap method.

CORRECT ANSWER:

```
public static void bubblesort (String[] array)
{
    int n = array.length;
    for(int pass=0; pass <= n-2; pass++)
    {
```

```

        for(int pos= 0; pos <= n-2-pass; pos++)
        {
            if (compareTo(array [pos], array [pos + 1])>0)
                swap (array, pos, pos + 1);
        }
    }
}

```

EXPLANATION:

We need to change the type of the array argument to be sorted to a String array, and then change the comparison operator. The method `compareTo` compares strings and returns a positive number if the first argument is greater than the second argument (i.e. are not in alphabetical order).

8. Give a version of the algorithm to sort an array of Student records (of type `Student`) in ascending order based on the mark field. You may assume you have an appropriate swap method.

CORRECT ANSWER:

```

public static void bubblesort (Student [] array)
{
    int n = array.length;
    for(int pass=0; pass <= n-2; pass++)
    {
        for(int pos= 0; pos <= n-2-pass; pos++)
        {
            if (array[pos].mark > array[pos + 1].mark)
                swap (array, pos, pos + 1);
        }
    }
}

```

EXPLANATION:

We compare the mark field but then swap the whole records round.